



GLOBAL RAIN

Practices for Secure Software Report

Table of Contents

DOCUMENT REVISION HISTORY	3
CLIENT.....	3
INSTRUCTIONS.....	ERROR! BOOKMARK NOT DEFINED.
DEVELOPER	3
1. ALGORITHM CIPHER	3
2. CERTIFICATE GENERATION	3
3. DEPLOY CIPHER.....	4
4. SECURE COMMUNICATIONS	4
5. SECONDARY TESTING.....	4
6. FUNCTIONAL TESTING	5
7. SUMMARY	5
8. INDUSTRY STANDARD BEST PRACTICES	6

Document Revision History

Version	Date	Author	Comments
1.0	February 20, 2026	Matthew Wood	

Client



Developer

Matthew Wood

1. Algorithm Cipher

For the file verification requirements of Artemis Financial, I recommend the **SHA-256** (Secure Hash Algorithm 256-bit) algorithm cipher.

- **Overview:** SHA-256 is a cryptographic hash function that generates a unique, fixed-size 256-bit (32-byte) signature for any given data input.
- **Hash Functions & Bit Levels:** As part of the SHA-2 family, it provides a significantly higher level of security and collision resistance than its predecessors, MD5 and SHA-1.
- **Keys and Randomness:** While hashing is a one-way process, the secure communication layer (HTTPS) utilizes **Asymmetric encryption** (RSA) for the initial handshake and **Symmetric encryption** (AES-256) for data transfer.
- **History & Current State:** SHA-256 is the current industry standard for data integrity and is required for compliance in financial and government sectors to prevent data tampering.

2. Certificate Generation

```
C:\Users\woodm\OneDrive\Documents\SNHU\CS-305\Module 7\CS 305 Project Two Code Base\ssl-server_student>keytool -export -alias artemis -file artemis.cer -keystore src/main/resources/keystore.p12
Enter keystore password:
Certificate stored in file <artemis.cer>
```



Certificate Information

This CA Root certificate is not trusted. To enable trust, install this certificate in the Trusted Root Certification Authorities store.

Issued to: Matthew Wood

Issued by: Matthew Wood

Valid from 2/22/2026 **to** 2/22/2027

3. Deploy Cipher

Data: Hello World Check Sum!

Name: Matthew Wood

SHA-256 Checksum: ab2aca08da294c82c67ae581bb5d309004220bece2ee07a84e13902029daa2cb

4. Secure Communications

ⓧ Not secure <https://localhost:8443/hash>

5. Secondary Testing

Project: ssl-server

com.snhu:ssl-server:0.0.1-SNAPSHOT

Scan Information ([show all](#)):

- dependency-check version: 12.2.0
- Report Generated On: Sun, 22 Feb 2026 20:54:13 -0600
- Dependencies Scanned: 44 (23 unique)
- Vulnerable Dependencies: 8
- Vulnerabilities Found: 42
- Vulnerabilities Suppressed: 0
- ...

Summary

Summary of Vulnerable Dependencies ([click to show all](#))

Dependency	Vulnerability IDs	Package	Highest Severity	CVE Count	Confidence	Evidence Count
jackson-databind-2.13.5.jar	cpe:2.3:a:fasterxml:jackson-databind:2.13.5:***** cpe:2.3:a:fasterxml:jackson-modules-java8:2.13.5:*****	pkg:maven/com.fasterxml.jackson.core/jackson-databind@2.13.5	MEDIUM	1	Highest	43
json-path-2.7.0.jar	cpe:2.3:a:json-path:jayway_jsonpath:2.7.0:*****	pkg:maven/com.jayway.jsonpath/json-path@2.7.0	MEDIUM	1	Highest	33
log4j-api-2.17.2.jar	cpe:2.3:a:apache:log4j:2.17.2:*****	pkg:maven/org.apache.logging.log4j/log4j-api@2.17.2	MEDIUM	1	Highest	44
logback-core-1.2.12.jar	cpe:2.3:a:qos:logback:1.2.12:*****	pkg:maven/ch.qos.logback/logback-core@1.2.12	HIGH	2	Highest	33
snakeyaml-1.30.jar	cpe:2.3:a:snakeyaml:project_snakeyaml:1.30:*****	pkg:maven/org.yaml/snakeyaml@1.30	CRITICAL	7	Highest	44
spring-core-5.3.31.jar	cpe:2.3:a:pivotal:software.spring.framework:5.3.31:***** cpe:2.3:a:springsource:spring_framework:5.3.31:***** cpe:2.3:a:vmware:spring_framework:5.3.31:*****	pkg:maven/org.springframework/spring-core@5.3.31	HIGH	3	Highest	37
spring-web-5.3.31.jar	cpe:2.3:a:pivotal:software.spring.framework:5.3.31:***** cpe:2.3:a:springsource:spring_framework:5.3.31:***** cpe:2.3:a:vmware:spring_framework:5.3.31:*****	pkg:maven/org.springframework/spring-web@5.3.31	CRITICAL	4	Highest	35
tomcat-embed-core-9.0.83.jar	cpe:2.3:a:apache:tomcat:9.0.83:***** cpe:2.3:a:apache_tomcat:apache_tomcat:9.0.83:*****	pkg:maven/org.apache.tomcat.embed/tomcat-embed-core@9.0.83	CRITICAL*	23	Highest	65

6. Functional Testing

```
INFO]
INFO] --- install:2.5.2:install (default-install) @ ssl-server ---
INFO] Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/sha
INFO] Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/sha
INFO] Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/sha
INFO] Downloading from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/p
INFO] Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/p
INFO] Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/sha
INFO] Installing C:\Users\woodm\OneDrive\Documents\SNHU\CS-305\Module 7\CS 305 Project Tw
INFO] Installing C:\Users\woodm\OneDrive\Documents\SNHU\CS-305\Module 7\CS 305 Project Tw
INFO] -----
INFO] BUILD SUCCESS
INFO] -----
INFO] Total time: 01:00 min
INFO] Finished at: 2026-02-22T20:54:13-06:00
INFO] -----
```

7. Summary

The refactoring of the Artemis Financial software application focused on adding critical layers of security to move from a vulnerable state to a production-quality integrated application. By following the **Vulnerability Assessment Process Flow**, I addressed several key security areas:

- **Cryptography and APIs:** I implemented the **SHA-256** cryptographic hash algorithm to create a secure data verification step, fulfilling the client's need for checksum functionality.
- **Client/Server:** I refactored the communication layer from HTTP to **HTTPS** by generating a self-signed certificate and configuring secure server properties, ensuring encrypted data transfer.

- **Code Review and Quality:** I used manual review and static testing to identify syntactical, logical, and security vulnerabilities, ensuring the code complies with security testing protocols.
- **Process:** My process involved an iterative design where I refactored the code base, conducted **Secondary Testing** with the **OWASP dependency-check tool**, and performed **Functional Testing** to confirm the application executed without errors.

8. Industry Standard Best Practices

I applied industry-standard best practices for secure coding to mitigate known vulnerabilities and ensure the long-term well-being of the organization:

- **Mitigation of Vulnerabilities:** By implementing **SHA-256** and **TLS encryption**, I ensured the application uses current encryption technologies to protect sensitive client data and financial information from unauthorized tampering or interception.
- **Security Maintenance:** I utilized the **dependency-check tool** to conduct a static security enhancement audit, identifying vulnerabilities in third-party libraries (such as Jackson-databind and SnakeYAML) that could compromise the system.
- **Organizational Value:** Applying these standards is essential for maintaining client trust and protecting Artemis Financial's reputation. It ensures compliance with financial data regulations and reduces the risk of data breaches, which is fundamental to Global Rain's mission that "Security is everyone's responsibility".